

Advanced Java Programming

Assignment No. 5 – Mini Projects (MVC Architecture)

This assignment covers 4 complete web application projects using Java EE technologies. Each project follows the **MVC (Model-View-Controller)** architecture pattern.

MVC Layer	Role	Technology Used
Model	Data & DB logic	JDBC / Hibernate Entity + DAO classes
View	UI & Display	JSP pages + HTML + CSS
Controller	Request handling	Servlet (HttpServletRequest doGet/doPost)
Service	Business logic	Spring IoC / plain Java service classes

Project 1: Student Record Management System

Technology Stack: JDBC + Servlet + JSP | MVC Architecture | MySQL

Project Structure:

```
StudentMVC/
|-- src/
|   |-- model/
|   |   |-- Student.java          (Model: POJO/Bean)
|   |   |-- StudentDAO.java      (Model: JDBC Data Access Object)
|   |   |-- DBConnection.java    (Model: DB connection utility)
|   |-- controller/
|   |   |-- StudentServlet.java  (Controller: handles all requests)
|-- WebContent/
|   |-- WEB-INF/
|   |   |-- web.xml
|   |-- views/
|   |   |-- studentList.jsp      (View: display all students)
|   |   |-- addStudent.jsp       (View: add new student form)
|   |   |-- editStudent.jsp      (View: edit existing student form)
|   |   |-- error.jsp            (View: error page)
|   |-- index.jsp                (View: welcome/redirect page)
```

SQL Setup:

```
CREATE DATABASE IF NOT EXISTS college_db;
USE college_db;

CREATE TABLE IF NOT EXISTS students (
    id      INT PRIMARY KEY AUTO_INCREMENT,
    name    VARCHAR(50) NOT NULL,
    age     INT         NOT NULL,
    email   VARCHAR(80) NOT NULL UNIQUE,
    course  VARCHAR(30) NOT NULL,
    phone   VARCHAR(15)
);

-- Sample data
INSERT INTO students VALUES (NULL, 'Rahul Sharma', 20, 'rahul@mail.com', 'BCA', '9876543210');
INSERT INTO students VALUES (NULL, 'Priya Patil', 21, 'priya@mail.com', 'MCA', '9123456789');
INSERT INTO students VALUES (NULL, 'Amit Joshi', 22, 'amit@mail.com', 'BSc CS', '9234567890');
```

1. Model — DBConnection.java:

```
package model;
import java.sql.*;

public class DBConnection {
    private static final String URL      = "jdbc:mysql://localhost:3306/college_db";
    private static final String USER     = "root";
    private static final String PASSWORD = "password";

    static {
        try { Class.forName("com.mysql.cj.jdbc.Driver"); }
        catch (ClassNotFoundException e) { throw new RuntimeException(e); }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

2. Model — Student.java (JavaBean / POJO):

```
package model;

public class Student {
    private int    id;
    private String name;
```



```

        deleteStudent(req, res); break;
    default:
        listStudents(req, res);
    }
} catch (Exception e) {
    req.setAttribute("error", e.getMessage());
    req.getRequestDispatcher("/views/error.jsp").forward(req, res);
}
}

@Override
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    String action = req.getParameter("action");
    try {
        if ("add".equals(action)) addStudent(req, res);
        else if ("update".equals(action)) updateStudent(req, res);
    } catch (Exception e) {
        req.setAttribute("error", e.getMessage());
        req.getRequestDispatcher("/views/error.jsp").forward(req, res);
    }
}

private void listStudents(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    List<Student> list = dao.getAllStudents();
    req.setAttribute("studentList", list);
    req.getRequestDispatcher("/views/studentList.jsp").forward(req, res);
}

private void showAddForm(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    req.getRequestDispatcher("/views/addStudent.jsp").forward(req, res);
}

private void showEditForm(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    int id = Integer.parseInt(req.getParameter("id"));
    Student s = dao.getStudentById(id);
    req.setAttribute("student", s);
    req.getRequestDispatcher("/views/editStudent.jsp").forward(req, res);
}

private void addStudent(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    // Form validation
    String name = req.getParameter("name").trim();
    String email = req.getParameter("email").trim();
    String course = req.getParameter("course").trim();
    int age = Integer.parseInt(req.getParameter("age"));
    String phone = req.getParameter("phone").trim();

    if (name.isEmpty() || email.isEmpty() || course.isEmpty()) {
        req.setAttribute("error", "All fields are required!");
        req.getRequestDispatcher("/views/addStudent.jsp").forward(req, res);
        return;
    }

    Student s = new Student(name, age, email, course, phone);
    boolean success = dao.addStudent(s);

    // Session message
    HttpSession session = req.getSession();
    session.setAttribute("message",
        success ? "Student added successfully!" : "Failed to add student.");
    res.sendRedirect("StudentServlet?action=list");
}

```

```

private void updateStudent(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    Student s = new Student();
    s.setId(Integer.parseInt(req.getParameter("id")));
    s.setName(req.getParameter("name").trim());
    s.setAge(Integer.parseInt(req.getParameter("age")));
    s.setEmail(req.getParameter("email").trim());
    s.setCourse(req.getParameter("course").trim());
    s.setPhone(req.getParameter("phone").trim());

    boolean success = dao.updateStudent(s);
    HttpSession session = req.getSession();
    session.setAttribute("message",
        success ? "Student updated successfully!" : "Update failed.");
    res.sendRedirect("StudentServlet?action=list");
}

private void deleteStudent(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    int id = Integer.parseInt(req.getParameter("id"));
    boolean success = dao.deleteStudent(id);
    HttpSession session = req.getSession();
    session.setAttribute("message",
        success ? "Student deleted!" : "Delete failed.");
    res.sendRedirect("StudentServlet?action=list");
}
}

```

5. View — studentList.jsp:

```

<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
    <style>
        body{font-family:Arial;margin:0;background:#f0f2f5}
        .header{background:#1a237e;color:white;padding:15px 30px;display:flex;
            justify-content:space-between;align-items:center}
        .container{max-width:1000px;margin:30px auto;background:white;
            border-radius:8px;padding:20px;box-shadow:0 2px 10px #ccc}
        table{width:100%;border-collapse:collapse}
        th{background:#1a237e;color:white;padding:10px;text-align:left}
        td{padding:9px;border-bottom:1px solid #eee}
        tr:hover{background:#e8eaf6}
        .btn{padding:5px 12px;border:none;border-radius:4px;cursor:pointer;
            text-decoration:none;font-size:13px}
        .btn-add{background:#4caf50;color:white}
        .btn-edit{background:#ff9800;color:white}
        .btn-del{background:#f44336;color:white}
        .msg-ok{background:#e8f5e9;border-left:4px solid #4caf50;padding:10px;margin-bottom:15px}
    </style>
</head>
<body>
<div class="header">
    <h2>Student Management System</h2>
    <a href="StudentServlet?action=new" class="btn btn-add">+ Add Student</a>
</div>
<div class="container">

    <!-- Session message -->
    <c:if test="${not empty sessionScope.message}">
        <div class="msg-ok"><b>${sessionScope.message}</b></div>
        <c:remove var="message" scope="session"/>
    </c:if>

    <h3>All Students (<c:out value="${studentList.size()}" /> records)</h3>

```

```

<table>
  <tr><th>ID</th><th>Name</th><th>Age</th><th>Email</th>
    <th>Course</th><th>Phone</th><th>Actions</th></tr>
  <c:choose>
    <c:when test="${not empty studentList}">
      <c:forEach var="s" items="${studentList}">
        <tr>
          <td>${s.id}</td>
          <td>${s.name}</td>
          <td>${s.age}</td>
          <td>${s.email}</td>
          <td>${s.course}</td>
          <td>${s.phone}</td>
          <td>
            <a href="StudentServlet?action=edit&id=${s.id}" class="btn btn-edit">Edit</a>
            <a href="StudentServlet?action=delete&id=${s.id}" class="btn btn-del"
              onclick="return confirm('Delete this student?')">Delete</a>
          </td>
        </tr>
      </c:forEach>
    </c:when>
    <c:otherwise>
      <tr><td colspan="7" style="text-align:center;color:grey">
        No students found. <a href="StudentServlet?action=new">Add one?</a>
      </td></tr>
    </c:otherwise>
  </c:choose>
</table>
</div>
</body>
</html>

```

6. View — addStudent.jsp:

```

<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head>
  <title>Add Student</title>
  <style>
    body{font-family:Arial;background:#f0f2f5;display:flex;
      justify-content:center;padding-top:40px}
    .form-box{background:white;padding:30px;border-radius:8px;
      width:420px;box-shadow:0 2px 10px #ccc}
    input,select{width:100%;padding:8px;margin:6px 0 14px;
      border:1px solid #ccc;border-radius:4px;box-sizing:border-box}
    .btn-sub{background:#1a237e;color:white;padding:10px;width:100%;
      border:none;border-radius:4px;cursor:pointer;font-size:15px}
    .err{color:red;font-size:13px}
  </style>
  <script>
    function validate() {
      var name=document.getElementById('name').value.trim();
      var email=document.getElementById('email').value.trim();
      var age=document.getElementById('age').value;
      if(!name){alert('Name required');return false;}
      if(age<1||age>100){alert('Invalid age');return false;}
      if(!email.includes('@')){alert('Invalid email');return false;}
      return true;
    }
  </script>
</head>
<body>
  <div class="form-box">
    <h2 style="color:#1a237e;text-align:center">Add New Student</h2>
    <c:if test="${not empty error}">
      <p class="err">${error}</p>
    </c:if>

```

```

<form action="StudentServlet" method="POST" onsubmit="return validate()">
  <input type="hidden" name="action" value="add">
  <label>Full Name *</label>
  <input type="text" id="name" name="name" required placeholder="Enter full name">
  <label>Age *</label>
  <input type="number" id="age" name="age" min="1" max="100" required>
  <label>Email *</label>
  <input type="email" id="email" name="email" required placeholder="email@example.com">
  <label>Course *</label>
  <select name="course" required>
    <option value="">-- Select Course --</option>
    <option>BCA</option><option>MCA</option>
    <option>BSc CS</option><option>BTech CS</option>
  </select>
  <label>Phone</label>
  <input type="text" name="phone" placeholder="10-digit phone number">
  <button type="submit" class="btn-sub">Add Student</button>
</form>
<p style="text-align:center;margin-top:10px">
  <a href="StudentServlet?action=list">Back to List</a>
</p>
</div>
</body>
</html>

```

7. View — editStudent.jsp:

```

<%@ page contentType="text/html;charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head><title>Edit Student</title>
<style>
  body{font-family:Arial;background:#f0f2f5;display:flex;
    justify-content:center;padding-top:40px}
  .form-box{background:white;padding:30px;border-radius:8px;
    width:420px;box-shadow:0 2px 10px #ccc}
  input,select{width:100%;padding:8px;margin:6px 0 14px;border:1px solid #ccc;
    border-radius:4px;box-sizing:border-box}
  .btn-upd{background:#ff9800;color:white;padding:10px;width:100%;
    border:none;border-radius:4px;cursor:pointer;font-size:15px}
</style>
</head>
<body>
<div class="form-box">
  <h2 style="color:#e65100;text-align:center">Edit Student</h2>
  <form action="StudentServlet" method="POST">
    <input type="hidden" name="action" value="update">
    <input type="hidden" name="id" value="{student.id}">

    <label>Full Name *</label>
    <input type="text" name="name" value="{student.name}" required>
    <label>Age *</label>
    <input type="number" name="age" value="{student.age}" required min="1" max="100">
    <label>Email *</label>
    <input type="email" name="email" value="{student.email}" required>
    <label>Course</label>
    <select name="course">
      <option ${student.course=='BCA'? 'selected':''}>BCA</option>
      <option ${student.course=='MCA'? 'selected':''}>MCA</option>
      <option ${student.course=='BSc CS'? 'selected':''}>BSc CS</option>
      <option ${student.course=='BTech CS'? 'selected':''}>BTech CS</option>
    </select>
    <label>Phone</label>
    <input type="text" name="phone" value="{student.phone}">

    <button type="submit" class="btn-upd">Update Student</button>
  </form>
  <p style="text-align:center"><a href="StudentServlet?action=list">Cancel</a></p>
</div>

```



```
</body>
</html>
```

8. web.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="4.0">

    <display-name>Student MVC</display-name>

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>controller.StudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/StudentServlet</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

    <error-page>
        <error-code>404</error-code>
        <location>/views/error.jsp</location>
    </error-page>

</web-app>
```

MVC Flow Diagram:

```
Browser
|
| HTTP Request (e.g., GET /StudentServlet?action=list)
v
StudentServlet.java (CONTROLLER)
|-- reads 'action' parameter
|-- calls StudentDAO.getAllStudents() -----> DBConnection + JDBC (MODEL)
|                                           (executes SELECT query)
|<-- receives List<Student>
|-- sets as request attribute: req.setAttribute("studentList", list)
|-- RequestDispatcher.forward() to studentList.jsp
v
studentList.jsp (VIEW)
|-- receives ${studentList} from request scope
|-- JSTL <c:forEach> iterates and builds HTML table
v
Browser displays HTML response
```

Project 2: Student Management System (SMS)

Technology Stack: Servlet + JSP + Hibernate + Spring | MVC + Service Layer

Project Structure:

```
SMS/
|-- src/
|   |-- model/
|   |   |-- Student.java          (Hibernate Entity)
|   |-- dao/
|   |   |-- StudentDAO.java       (Interface)
|   |   |-- StudentDAOImpl.java   (Hibernate DAO Implementation)
|   |-- service/
|   |   |-- StudentService.java   (Interface)
|   |   |-- StudentServiceImpl.java (Spring Service Bean)
|   |-- controller/
|   |   |-- StudentController.java (Servlet / Controller)
|   |-- util/
|   |   |-- HibernateUtil.java    (SessionFactory)
|-- WebContent/
|   |-- WEB-INF/
|   |   |-- applicationContext.xml (Spring config)
|   |   |-- hibernate.cfg.xml
|   |   |-- web.xml
|   |-- views/
|   |   |-- students.jsp, addForm.jsp, editForm.jsp, login.jsp
```

1. Model — Student.java (Hibernate Entity):

```
package model;
import javax.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "name", nullable = false, length = 50)
    private String name;

    @Column(name = "age")
    private int age;

    @Column(name = "email", unique = true, length = 80)
    private String email;

    @Column(name = "course", length = 30)
    private String course;

    @Column(name = "phone", length = 15)
    private String phone;

    public Student() {}
    public Student(String name, int age, String email, String course, String phone) {
        this.name=name; this.age=age; this.email=email;
        this.course=course; this.phone=phone;
    }

    // Getters and Setters (same as Project 1 Student class)
    public int getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getEmail() { return email; }
    public String getCourse() { return course; }
```

```

    public String getPhone() { return phone; }
    public void setId(int id) { this.id=id; }
    public void setName(String n) { this.name=n; }
    public void setAge(int a) { this.age=a; }
    public void setEmail(String e) { this.email=e; }
    public void setCourse(String c) { this.course=c; }
    public void setPhone(String p) { this.phone=p; }
}

```

2. DAO Interface — StudentDAO.java:

```

package dao;
import model.Student;
import java.util.List;

public interface StudentDAO {
    void save(Student student);
    Student findById(int id);
    List<Student> findAll();
    List<Student> searchByName(String name);
    void update(Student student);
    void delete(int id);
}

```

3. DAO Implementation — StudentDAOImpl.java (Hibernate):

```

package dao;
import model.Student;
import org.hibernate.*;
import util.HibernateUtil;
import java.util.List;

public class StudentDAOImpl implements StudentDAO {

    @Override
    public void save(Student student) {
        Session s = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = s.beginTransaction();
        try {
            s.save(student);
            tx.commit();
        } catch (Exception e) {
            tx.rollback(); throw e;
        } finally { s.close(); }
    }

    @Override
    public Student findById(int id) {
        Session s = HibernateUtil.getSessionFactory().openSession();
        try { return s.get(Student.class, id); }
        finally { s.close(); }
    }

    @Override
    public List<Student> findAll() {
        Session s = HibernateUtil.getSessionFactory().openSession();
        try {
            return s.createQuery("FROM Student ORDER BY name", Student.class)
                .getResultList();
        } finally { s.close(); }
    }

    @Override
    public List<Student> searchByName(String name) {
        Session s = HibernateUtil.getSessionFactory().openSession();
        try {
            return s.createQuery(
                "FROM Student WHERE name LIKE :n ORDER BY name", Student.class)
                .setParameter("n", "%" + name + "%")
                .getResultList();
        }
    }
}

```

```

        } finally { s.close(); }
    }

    @Override
    public void update(Student student) {
        Session s = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = s.beginTransaction();
        try { s.update(student); tx.commit(); }
        catch (Exception e) { tx.rollback(); throw e; }
        finally { s.close(); }
    }

    @Override
    public void delete(int id) {
        Session s = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = s.beginTransaction();
        try {
            Student student = s.get(Student.class, id);
            if (student != null) s.delete(student);
            tx.commit();
        } catch (Exception e) { tx.rollback(); throw e; }
        finally { s.close(); }
    }
}

```

4. Service Layer — StudentServiceImpl.java (Spring Bean):

```

package service;
import dao.StudentDAO;
import model.Student;
import java.util.List;

// This class is a Spring Bean – injected with StudentDAO by Spring
public class StudentServiceImpl implements StudentService {

    // Injected by Spring (setter injection in applicationContext.xml)
    private StudentDAO studentDAO;

    public void setStudentDAO(StudentDAO studentDAO) {
        this.studentDAO = studentDAO;
    }

    @Override
    public void addStudent(Student student) {
        // Business logic: validation before saving
        if (student.getName() == null || student.getName().isEmpty())
            throw new IllegalArgumentException("Student name cannot be empty!");
        if (student.getAge() < 15 || student.getAge() > 60)
            throw new IllegalArgumentException("Invalid age: " + student.getAge());
        studentDAO.save(student);
    }

    @Override
    public Student getStudent(int id) {
        Student s = studentDAO.findById(id);
        if (s == null) throw new RuntimeException("Student not found: ID=" + id);
        return s;
    }

    @Override
    public List<Student> getAllStudents() { return studentDAO.findAll(); }

    @Override
    public List<Student> searchStudents(String name) { return studentDAO.searchByName(name); }

    @Override
    public void updateStudent(Student student) { studentDAO.update(student); }

    @Override

```

```

        public void deleteStudent(int id)                { studentDAO.delete(id); }
    }

```

5. Spring Config — applicationContext.xml (WEB-INF):

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- DAO Bean -->
    <bean id="studentDAO" class="dao.StudentDAOImpl" />

    <!-- Service Bean with DAO injected via setter -->
    <bean id="studentService" class="service.StudentServiceImpl">
        <property name="studentDAO" ref="studentDAO" />
    </bean>

</beans>

```

6. Controller — StudentController.java (Servlet):

```

package controller;
import model.Student;
import service.StudentService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
import java.io.*;
import java.util.List;

@WebServlet("/student/*")
public class StudentController extends HttpServlet {

    private StudentService studentService;

    @Override
    public void init() {
        // Load Spring context and get Service bean
        ApplicationContext ctx = new ClassPathXmlApplicationContext(
            "/WEB-INF/applicationContext.xml");
        studentService = (StudentService) ctx.getBean("studentService");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String path = req.getPathInfo();
        if (path == null) path = "/list";

        switch (path) {
            case "/list":    listStudents(req, res);    break;
            case "/new":     showAddForm(req, res);     break;
            case "/edit":    showEditForm(req, res);    break;
            case "/delete":  deleteStudent(req, res);  break;
            case "/search":  searchStudents(req, res);  break;
            default:         listStudents(req, res);
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String path = req.getPathInfo();
        if ("/add".equals(path))    addStudent(req, res);
        else if ("/update".equals(path)) updateStudent(req, res);
    }
}

```

```

private void listStudents(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    req.setAttribute("students", studentService.getAllStudents());
    req.getRequestDispatcher("/views/students.jsp").forward(req, res);
}

private void showAddForm(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    req.getRequestDispatcher("/views/addForm.jsp").forward(req, res);
}

private void showEditForm(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    int id = Integer.parseInt(req.getParameter("id"));
    req.setAttribute("student", studentService.getStudent(id));
    req.getRequestDispatcher("/views/editForm.jsp").forward(req, res);
}

private void addStudent(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    try {
        Student s = buildStudentFromRequest(req);
        studentService.addStudent(s);
        req.getSession().setAttribute("msg", "Student added!");
    } catch (Exception e) {
        req.getSession().setAttribute("msg", "Error: " + e.getMessage());
    }
    res.sendRedirect(req.getContextPath() + "/student/list");
}

private void updateStudent(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    Student s = buildStudentFromRequest(req);
    s.setId(Integer.parseInt(req.getParameter("id")));
    studentService.updateStudent(s);
    req.getSession().setAttribute("msg", "Student updated!");
    res.sendRedirect(req.getContextPath() + "/student/list");
}

private void deleteStudent(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    studentService.deleteStudent(Integer.parseInt(req.getParameter("id")));
    req.getSession().setAttribute("msg", "Student deleted!");
    res.sendRedirect(req.getContextPath() + "/student/list");
}

private void searchStudents(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    String q = req.getParameter("q");
    req.setAttribute("students", studentService.searchStudents(q));
    req.setAttribute("searchTerm", q);
    req.getRequestDispatcher("/views/students.jsp").forward(req, res);
}

private Student buildStudentFromRequest(HttpServletRequest req) {
    return new Student(
        req.getParameter("name").trim(),
        Integer.parseInt(req.getParameter("age")),
        req.getParameter("email").trim(),
        req.getParameter("course").trim(),
        req.getParameter("phone").trim()
    );
}
}

```

Architecture Flow — Project 2:

Browser Request

|

```

    v
StudentController (Servlet – CONTROLLER)
| Spring injects StudentService via init()
|
v
StudentServiceImpl (Spring Bean – SERVICE LAYER)
| Business logic + validation
| Spring injects StudentDAO
|
v
StudentDAOImpl (Hibernate DAO – MODEL)
| Hibernate SessionFactory
|
v
MySQL Database (students table)
|
v (results flow back up through layers)
v
students.jsp / addForm.jsp / editForm.jsp (JSP – VIEW)
|
v
Browser Response (HTML)

```

Project 3: Employee Management System (EMS)

Technology Stack: Servlet + JSP + Hibernate + Spring | Search by Department

SQL Setup:

```
CREATE TABLE employees (  
    id            INT PRIMARY KEY AUTO_INCREMENT,  
    name          VARCHAR(60) NOT NULL,  
    email         VARCHAR(80) UNIQUE NOT NULL,  
    department    VARCHAR(40) NOT NULL,  
    designation   VARCHAR(50),  
    salary        DOUBLE,  
    phone         VARCHAR(15),  
    join_date     DATE  
);  
  
INSERT INTO employees VALUES  
(NULL, 'Rajesh Kumar', 'rajesh@co.com', 'IT', 'Senior Developer', 75000, '9876543210', '2022-01-15'),  
(NULL, 'Sunita Sharma', 'sunita@co.com', 'HR', 'HR Manager', 60000, '9123456789', '2021-06-01'),  
(NULL, 'Amit Verma', 'amit@co.com', 'Finance', 'Accountant', 55000, '9234567890', '2023-03-10');
```

1. Model — Employee.java (Hibernate Entity):

```
package model;  
import javax.persistence.*;  
import java.util.Date;  
  
@Entity  
@Table(name = "employees")  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private int id;  
  
    @Column(name = "name", nullable = false, length = 60)  
    private String name;  
  
    @Column(name = "email", unique = true, length = 80)  
    private String email;  
  
    @Column(name = "department", nullable = false, length = 40)  
    private String department;  
  
    @Column(name = "designation", length = 50)  
    private String designation;  
  
    @Column(name = "salary")  
    private double salary;  
  
    @Column(name = "phone", length = 15)  
    private String phone;  
  
    @Column(name = "join_date")  
    @Temporal(TemporalType.DATE)  
    private Date joinDate;  
  
    public Employee() {}  
  
    // Getters  
    public int getId() { return id; }  
    public String getName() { return name; }  
    public String getEmail() { return email; }  
    public String getDepartment() { return department; }  
    public String getDesignation() { return designation; }  
    public double getSalary() { return salary; }  
    public String getPhone() { return phone; }
```



```

<div class="header">
  <h2>Employee Management System</h2>
</div>
<div class="container">

  <!-- Filter Bar -->
  <div class="filter-bar">
    <form action="EmployeeController" method="GET" style="display:flex;gap:8px">
      <input type="hidden" name="action" value="filterDept">
      <select name="department">
        <option value="">All Departments</option>
        <option value="IT"      ${param.department=='IT'? 'selected':''}>IT</option>
        <option value="HR"      ${param.department=='HR'? 'selected':''}>HR</option>
        <option value="Finance" ${param.department=='Finance'? 'selected':''}>Finance</option>
        <option value="Admin"   ${param.department=='Admin'? 'selected':''}>Admin</option>
      </select>
      <button type="submit" class="btn btn-grn">Filter</button>
      <a href="EmployeeController?action=list" class="btn" style="background:#757575;color:white">Reset</a>
    </form>
    <a href="EmployeeController?action=new" class="btn btn-grn" style="margin-left:auto">+ Add Employee</a>
  </div>

  <!-- Success message -->
  <c:if test="${not empty sessionScope.msg}">
    <div style="background:#e8f5e9;padding:10px;border-left:4px solid #4caf50;margin-bottom:10px">
      ${sessionScope.msg}
    </div>
    <c:remove var="msg" scope="session"/>
  </c:if>

  <table>
    <tr>
      <th>ID</th><th>Name</th><th>Email</th><th>Department</th>
      <th>Designation</th><th>Salary</th><th>Actions</th>
    </tr>
    <c:forEach var="emp" items="${employees}">
      <tr>
        <td>${emp.id}</td>
        <td>${emp.name}</td>
        <td>${emp.email}</td>
        <td><span class="dept-badge">${emp.department}</span></td>
        <td>${emp.designation}</td>
        <td><fmt:formatNumber value="${emp.salary}" type="currency" currencySymbol="Rs." /></td>
        <td>
          <a href="EmployeeController?action=edit&id=${emp.id}" class="btn btn-org">Edit</a>
          <a href="EmployeeController?action=delete&id=${emp.id}" class="btn btn-red"
            onclick="return confirm('Delete this employee?')">Del</a>
        </td>
      </tr>
    </c:forEach>
  </table>
  <p><b>Total: ${employees.size()} employee(s)</b></p>
</div>
</body>
</html>

```

Project 4: Online Library Management System

Technology Stack: Servlet + JSP + Hibernate + Spring | Books + Issue + Return

SQL Schema:

```
CREATE TABLE books (
    id            INT PRIMARY KEY AUTO_INCREMENT,
    title         VARCHAR(100) NOT NULL,
    author        VARCHAR(60)  NOT NULL,
    isbn          VARCHAR(20)  UNIQUE,
    category       VARCHAR(40),
    total_copies  INT DEFAULT 1,
    available     INT DEFAULT 1
);

CREATE TABLE issue_records (
    id            INT PRIMARY KEY AUTO_INCREMENT,
    book_id       INT NOT NULL,
    borrower_name VARCHAR(60),
    borrower_id   VARCHAR(20),
    issue_date    DATE NOT NULL,
    return_date   DATE,
    status        ENUM('ISSUED', 'RETURNED') DEFAULT 'ISSUED',
    FOREIGN KEY (book_id) REFERENCES books(id)
);

INSERT INTO books VALUES
(NULL, 'Core Java', 'Cay Horstmann', '978-01', 'Java', 3, 3),
(NULL, 'Spring in Action', 'Craig Walls', '978-02', 'Spring', 2, 2),
(NULL, 'Design Patterns', 'GoF', '978-03', 'CS', 1, 1);
```

1. Model — Book.java (Hibernate Entity):

```
package model;
import javax.persistence.*;
import java.util.List;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "title", nullable = false, length = 100)
    private String title;

    @Column(name = "author", nullable = false, length = 60)
    private String author;

    @Column(name = "isbn", unique = true, length = 20)
    private String isbn;

    @Column(name = "category", length = 40)
    private String category;

    @Column(name = "total_copies")
    private int totalCopies;

    @Column(name = "available")
    private int available;

    @OneToMany(mappedBy = "book", cascade = CascadeType.ALL)
    private List<IssueRecord> issueRecords;

    public Book() {}
```

```

    public boolean isAvailable() { return available > 0; }

    // Getters
    public int getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public String getIsbn() { return isbn; }
    public String getCategory() { return category; }
    public int getTotalCopies() { return totalCopies; }
    public int getAvailable() { return available; }

    // Setters
    public void setId(int id) { this.id = id; }
    public void setTitle(String title) { this.title = title; }
    public void setAuthor(String author) { this.author = author; }
    public void setIsbn(String isbn) { this.isbn = isbn; }
    public void setCategory(String category) { this.category = category; }
    public void setTotalCopies(int t) { this.totalCopies = t; }
    public void setAvailable(int a) { this.available = a; }
}

```

2. Model — IssueRecord.java (Hibernate Entity):

```

package model;
import javax.persistence.*;
import java.util.Date;

@Entity
@Table(name = "issue_records")
public class IssueRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @ManyToOne
    @JoinColumn(name = "book_id", nullable = false)
    private Book book;

    @Column(name = "borrower_name", length = 60)
    private String borrowerName;

    @Column(name = "borrower_id", length = 20)
    private String borrowerId;

    @Column(name = "issue_date")
    @Temporal(TemporalType.DATE)
    private Date issueDate;

    @Column(name = "return_date")
    @Temporal(TemporalType.DATE)
    private Date returnDate;

    @Column(name = "status")
    @Enumerated(EnumType.STRING)
    private Status status = Status.ISSUED;

    public enum Status { ISSUED, RETURNED }

    public IssueRecord() {}

    // Getters
    public int getId() { return id; }
    public Book getBook() { return book; }
    public String getBorrowerName() { return borrowerName; }
    public String getBorrowerId() { return borrowerId; }
    public Date getIssueDate() { return issueDate; }
    public Date getReturnDate() { return returnDate; }
    public Status getStatus() { return status; }
}

```

```
// Setters
public void setId(int id)           { this.id = id; }
public void setBook(Book book)     { this.book = book; }
public void setBorrowerName(String name) { this.borrowerName = name; }
public void setBorrowerId(String bid) { this.borrowerId = bid; }
public void setIssueDate(Date issueDate) { this.issueDate = issueDate; }
public void setReturnDate(Date returnDate) { this.returnDate = returnDate; }
public void setStatus(Status status) { this.status = status; }
}
```

3. Service — LibraryServiceImpl.java (Spring Bean — Core Logic):

[illegible][illegible]

```
@Override
public void issueBook(int bookId, String borrowerName, String borrowerId) {
    Book book = bookDAO.findById(bookId);
    if (book == null)
        throw new RuntimeException("Book not found!");
    if (!book.isAvailable())
        throw new RuntimeException("No copies available for: " + book.getTitle());
}
```

```
// Decrease available count
book.setAvailable(book.getAvailable() - 1);
bookDAO.update(book);
```

```
// Create issue record
IssueRecord record = new IssueRecord();
record.setBook(book);
record.setBorrowerName(borrowerName);
record.setBorrowerId(borrowerId);
record.setIssueDate(new Date());
record.setStatus(IssueRecord.Status.ISSUED);
issueDAO.save(record);
```

}

[illegible]

```
@Override
public void returnBook(int issueRecordId) {
    IssueRecord record = issueDAO.findById(issueRecordId);
    if (record == null)
        throw new RuntimeException("Issue record not found!");
    if (record.getStatus() == IssueRecord.Status.RETURNED)
        throw new RuntimeException("Book already returned!");
}
```

```
// Update record
record.setReturnDate(new Date());
```

```

        record.setStatus(IssueRecord.Status.RETURNED);
        issueDAO.update(record);

        // Increase available count
        Book book = record.getBook();
        book.setAvailable(book.getAvailable() + 1);
        bookDAO.update(book);
    }

    @Override public List<Book> getAllBooks() { return bookDAO.findAll(); }
    @Override public Book getBook(int id) { return bookDAO.findById(id); }
    @Override public void deleteBook(int id) { bookDAO.delete(id); }
    @Override public List<IssueRecord> getAllIssues() { return issueDAO.findAll(); }
    @Override public List<IssueRecord> getActiveIssues() {
        return issueDAO.findByStatus(IssueRecord.Status.ISSUED);
    }
}

```

4. Spring Config — applicationContext.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- DAO Beans -->
    <bean id="bookDAO" class="dao.BookDAOImpl" />
    <bean id="issueDAO" class="dao.IssueDAOImpl" />

    <!-- Service Bean — both DAOs injected -->
    <bean id="libraryService" class="service.LibraryServiceImpl">
        <property name="bookDAO" ref="bookDAO" />
        <property name="issueDAO" ref="issueDAO" />
    </bean>

</beans>

```

5. Controller — LibraryController.java (Servlet):

```

package controller;
import model.*;
import service.LibraryService;
import org.springframework.context.*;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/library/*")
public class LibraryController extends HttpServlet {

    private LibraryService libraryService;

    @Override
    public void init() {
        ApplicationContext ctx = new ClassPathXmlApplicationContext(
            "/WEB-INF/applicationContext.xml");
        libraryService = (LibraryService) ctx.getBean("libraryService");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String path = req.getPathInfo();
        if (path == null) path = "/books";
        try {
            switch (path) {
                case "/books":

```

```

        req.setAttribute("books", libraryService.getAllBooks());
        forward(req, res, "/views/books.jsp"); break;
    case "/new":
        forward(req, res, "/views/addBook.jsp"); break;
    case "/issues":
        req.setAttribute("issues", libraryService.getAllIssues());
        forward(req, res, "/views/issues.jsp"); break;
    case "/issueForm":
        req.setAttribute("books", libraryService.getAllBooks());
        forward(req, res, "/views/issueForm.jsp"); break;
    case "/delete":
        libraryService.deleteBook(Integer.parseInt(req.getParameter("id")));
        msg(req, "Book deleted!"); res.sendRedirect(base(req)+"/library/books"); break;
    case "/return":
        libraryService.returnBook(Integer.parseInt(req.getParameter("id")));
        msg(req, "Book returned!"); res.sendRedirect(base(req)+"/library/issues"); break;
    }
} catch (Exception e) {
    msg(req, "Error: " + e.getMessage());
    res.sendRedirect(base(req)+"/library/books");
}
}

@Override
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    String path = req.getPathInfo();
    try {
        if ("/addBook".equals(path)) {
            Book b = new Book();
            b.setTitle(req.getParameter("title").trim());
            b.setAuthor(req.getParameter("author").trim());
            b.setIsbn(req.getParameter("isbn").trim());
            b.setCategory(req.getParameter("category").trim());
            b.setTotalCopies(Integer.parseInt(req.getParameter("copies")));
            libraryService.addBook(b);
            msg(req, "Book added successfully!");
        } else if ("/issueBook".equals(path)) {
            libraryService.issueBook(
                Integer.parseInt(req.getParameter("bookId")),
                req.getParameter("borrowerName").trim(),
                req.getParameter("borrowerId").trim());
            msg(req, "Book issued successfully!");
        }
    } catch (Exception e) {
        msg(req, "Error: " + e.getMessage());
    }
    res.sendRedirect(base(req)+"/library/books");
}

private void forward(HttpServletRequest req, HttpServletResponse res, String page)
    throws ServletException, IOException {
    req.getRequestDispatcher(page).forward(req, res);
}

private void msg(HttpServletRequest req, String m) {
    req.getSession().setAttribute("msg", m);
}

private String base(HttpServletRequest req) {
    return req.getContextPath();
}
}

```

6. View — books.jsp:

```

<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head>

```



```

<title>Library Management</title>
<style>
  body{font-family:Arial;margin:0;background:#fafafa}
  .header{background:#4a148c;color:white;padding:15px 30px;
    display:flex;justify-content:space-between;align-items:center}
  nav a{color:white;margin-left:15px;text-decoration:none;font-size:14px}
  .container{max-width:1050px;margin:25px auto;background:white;
    padding:20px;border-radius:8px;box-shadow:0 2px 8px #ddd}
  table{width:100%;border-collapse:collapse}
  th{background:#4a148c;color:white;padding:9px}
  td{padding:8px;border-bottom:1px solid #eee;text-align:center}
  .badge-avail{background:#e8f5e9;color:#1b5e20;padding:3px 8px;border-radius:10px}
  .badge-out {background:#ffebee;color:#b71c1c;padding:3px 8px;border-radius:10px}
  .btn{padding:5px 12px;border-radius:4px;border:none;cursor:pointer;
    text-decoration:none;font-size:12px}
  .msg{background:#f3e5f5;border-left:4px solid #7b1fa2;padding:10px;margin-bottom:15px}
</style>
</head>
<body>
<div class="header">
  <h2>Online Library Management</h2>
  <nav>
    <a href="books">All Books</a>
    <a href="issues">Issue Records</a>
    <a href="issueForm">Issue a Book</a>
    <a href="new">Add Book</a>
  </nav>
</div>
<div class="container">
  <c:if test="{not empty sessionScope.msg}">
    <div class="msg">{sessionScope.msg}</div>
    <c:remove var="msg" scope="session"/>
  </c:if>

  <h3>All Books ({books.size()}) total</h3>
  <table>
    <tr>
      <th>ID</th><th>Title</th><th>Author</th><th>ISBN</th>
      <th>Category</th><th>Total</th><th>Available</th><th>Actions</th>
    </tr>
    <c:forEach var="book" items="{books}">
      <tr>
        <td>{book.id}</td>
        <td style="text-align:left"><b>{book.title}</b></td>
        <td>{book.author}</td>
        <td>{book.isbn}</td>
        <td>{book.category}</td>
        <td>{book.totalCopies}</td>
        <td>
          <c:choose>
            <c:when test="{book.available > 0}">
              <span class="badge-avail">{book.available} Available</span>
            </c:when>
            <c:otherwise>
              <span class="badge-out">All Issued</span>
            </c:otherwise>
          </c:choose>
        </td>
        <td>
          <a href="delete?id={book.id}" class="btn"
            style="background:#c62828;color:white"
            onclick="return confirm('Delete this book?')">Delete</a>
        </td>
      </tr>
    </c:forEach>
  </table>
</div>
</body>

```

</html>

7. View — issueForm.jsp (Issue/Return Book Form):

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head><title>Issue Book</title>
<style>
  body{font-family:Arial;display:flex;justify-content:center;
    background:#f3e5f5;padding-top:40px}
  .box{background:white;padding:30px;border-radius:8px;
    width:420px;box-shadow:0 2px 10px #ccc}
  select,input{width:100%;padding:8px;margin:6px 0 14px;
    border:1px solid #ccc;border-radius:4px;box-sizing:border-box}
  .btn-sub{background:#4a148c;color:white;padding:10px;width:100%;
    border:none;border-radius:4px;cursor:pointer;font-size:15px}
</style>
</head>
<body>
<div class="box">
  <h2 style="color:#4a148c;text-align:center">Issue a Book</h2>
  <form action="issueBook" method="POST">
    <label>Select Book *</label>
    <select name="bookId" required>
      <option value="">-- Choose Book --</option>
      <c:forEach var="b" items="${books}">
        <c:if test="${b.available > 0}">
          <option value="${b.id}">${b.title} (${b.available} available)</option>
        </c:if>
      </c:forEach>
    </select>
    <label>Borrower Name *</label>
    <input type="text" name="borrowerName" required placeholder="Student / Staff name">
    <label>Borrower ID (Roll No / Emp ID) *</label>
    <input type="text" name="borrowerId" required placeholder="e.g., MCA2023001">
    <button type="submit" class="btn-sub">Issue Book</button>
  </form>
  <p style="text-align:center"><a href="books">Back to Books</a></p>
</div>
</body>
</html>
```

Complete Project 4 — Data Flow:

```
User Action: "Issue a Book"
|
| POST /library/issueBook
v
LibraryController.doPost()      [Controller]
|
| calls libraryService.issueBook(bookId, name, id)
v
LibraryServiceImpl              [Spring Service – Business Logic]
| 1. Check if book exists
| 2. Check if copies available
| 3. Decrease book.available by 1
| 4. Create IssueRecord with date + ISSUED status
|
| calls bookDAO.update(book) + issueDAO.save(record)
v
BookDAOImpl + IssueDAOImpl      [Hibernate DAO – Model]
| Hibernate generates:
| UPDATE books SET available = ? WHERE id = ?
| INSERT INTO issue_records (book_id, borrower_name, ...) VALUES (?, ?, ...)
v
MySQL Database

Return Action ("Return Book"):
```

```
| issueDAO.findById(id)
| record.setStatus(RETURNED) + record.setReturnDate(today)
| book.setAvailable(available + 1)
v
DB updated + success message shown in JSP
```